Task 1: Representation Learning

1 Introduction

Many real world datasets contain a large number of features. For example, a small grayscale image of size 28×28 already contains 784 pixel values. When the number of dimensions becomes large, it becomes harder to analyze or visualize the data.

Because of this, machine learning uses methods that compress data into a smaller representation while still keeping the important information. These compressed representations are called **latent representations**.

This report studies three main ideas:

- Autoencoders for learning compressed representations
- Variational Autoencoders (VAEs) which extend autoencoders using probability
- Visualization techniques such as PCA, t-SNE, and UMAP

2 Autoencoders

An autoencoder is a neural network that tries to reproduce its input.

It has two parts:

- Encoder
- Decoder

The encoder compresses the input x into a smaller representation z:

$$z = f(x) \tag{1}$$

The decoder then reconstructs the input:

$$\hat{x} = g(z) \tag{2}$$

So the full process is:

$$x \to z \to \hat{x}$$

The vector z is called the **latent vector**. It contains a compressed summary of the original data.

3 Loss Functions for Autoencoders

To train the autoencoder we measure how close the reconstructed output $\hat{x}$ is to the original input x.

3.1 Mean Squared Error

For continuous data we usually use Mean Squared Error (MSE):

$$L = \frac{1}{n}\sum_{i=1}^{n}(x_i - \hat{x}_i)^2 \tag{3}$$

This loss simply measures the squared difference between the original values and the reconstructed values.

If the reconstruction is good, the loss becomes small.

3.2 Binary Cross Entropy

For binary data or normalized pixel values we often use Binary Cross Entropy:

$$L = -\sum_{i=1}^{n}[x_i \log(\hat{x}_i) + (1 - x_i)\log(1 - \hat{x}_i)] \tag{4}$$

This loss treats the output as probabilities and measures how well the predictions match the real values.

4 Transition from Autoencoders to Variational Autoencoders

Standard autoencoders work well for compression, but they have an important limitation. The latent space they learn is often irregular and difficult to use for generating new data.

consider the standard encoder:

$$z = f(x) \tag{5}$$

This means each input x is mapped to a single point in latent space. Two similar inputs might end up in very different locations, which makes it difficult to generate new meaningful samples.

Variational Autoencoders solve this problem by making the latent representation probabilistic instead of deterministic.

4.1 Step 1: Representing Latent Variables as Distributions

Instead of mapping each input to a single vector, the encoder outputs two values:

- Mean $\mu(x)$

- Standard deviation $\sigma(x)$

These define a Gaussian distribution:

$$q(z|x) = \mathcal{N}(\mu(x), \sigma^2(x)) \tag{6}$$

Instead of using a fixed vector z, we sample:

$$z \sim q(z|x) \tag{7}$$

This means every input is represented by a distribution rather than a single point.

4.2 Step 2: Introducing a Generative Model

In a VAE we assume that data is generated from hidden variables.

First we sample a latent variable from a prior distribution:

$$z \sim p(z) \tag{8}$$

Usually the prior is chosen as a standard normal distribution:

$$p(z) = \mathcal{N}(0, 1)$$

Then the data is generated using a decoder:

$$x \sim p(x|z) \tag{9}$$

The decoder network learns this distribution.

4.3 Step 3: Deriving the VAE Objective

The goal is to maximize the probability of the observed data:

$$\log p(x) \tag{10}$$

computing this requires integrating over all possible values of z:

$$p(x) = \int p(x|z)p(z)dz \tag{11}$$

This integral is usually impossible to compute directly.

To solve this problem we introduce an approximate distribution $q(z|x)$ and derive the Evidence Lower Bound (ELBO):

$$ELBO = E_{q(z|x)}[\log p(x|z)] - KL(q(z|x)||p(z)) \tag{12}$$

This equation contains two important parts.

Reconstruction term

$$E_{q(z|x)}[\log p(x|z)]$$

This term ensures that the reconstructed output is similar to the original input.

Regularization term

$$KL(q(z|x)||p(z))$$

This term forces the latent distribution to stay close to the prior distribution.

Together these terms ensure that the latent space becomes smooth and structured.

4.4 Reparameterization Trick

Sampling directly from a distribution breaks backpropagation. To solve this we rewrite the sampling step.

$$z = \mu + \sigma\epsilon \tag{13}$$

where

$$\epsilon \sim \mathcal{N}(0, 1)$$

This separates randomness from the learnable parameters and allows gradients to flow during training.

5 Latent Space Visualization

High dimensional datasets are difficult to visualize. Dimensionality reduction techniques help project this data into two or three dimensions.

The most common methods are PCA, t-SNE and UMAP.

6 Principal Component Analysis (PCA)

PCA is a linear dimensionality reduction technique. It finds directions where the data has the highest variance.

First we compute the covariance matrix:

$$C = \frac{1}{n} X^T X \tag{14}$$

Then we compute eigenvectors and eigenvalues:

$$Cv = \lambda v \tag{15}$$

The eigenvectors give the principal directions of the data.

The data can then be projected onto these directions to reduce the number of dimensions.

7 t-SNE

t-SNE is a nonlinear technique mainly used for visualization.

The main idea is to keep similar points close together in the low dimensional space.

The algorithm converts pairwise distances into probabilities and then minimizes the difference between the high dimensional and low dimensional distributions using KL divergence.

t-SNE is very good at showing cluster structures.

8 UMAP

UMAP is another nonlinear dimensionality reduction method.

It assumes that the data lies on a low dimensional manifold inside a higher dimensional space.

UMAP first builds a nearest neighbor graph and then learns a lower dimensional representation that preserves this structure.

Compared to t-SNE, UMAP is often faster and preserves more global structure.

9 Comparison of Methods

Method	Linearity	Structure	Cost	Use Case
PCA	Linear	Global variance	Low	Compression
t-SNE	Nonlinear	Local clusters	High	Visualization
UMAP	Nonlinear	Local + global	Medium	Large datasets

10 Decision Guide: Choosing the Right Algorithm

Choosing the correct dimensionality reduction method depends on the dataset size and the structure of the data.

10.1 When to Use PCA

PCA works best when the dataset has mostly linear relationships.

Use PCA when:

- The dataset is very large
- The goal is simple dimensionality reduction
- Speed is important
- The data structure is roughly linear

PCA is commonly used as a preprocessing step before other machine learning algorithms.

10.2 When to Use t-SNE

t-SNE is mainly used for visualizing clusters.

Use t-SNE when:

- The dataset size is small or medium
- The goal is visualization
- Local cluster structure is important
- You want to clearly see separate groups in the data

However t-SNE can be slow on very large datasets.

10.3 When to Use UMAP

UMAP is often preferred when working with large datasets.

Use UMAP when:

- The dataset is large
- Both local and global structure are important
- You need faster computation than t-SNE
- You want stable embeddings

UMAP is commonly used for large biological datasets and large image datasets.

Summary

- PCA: fast and simple, good for preprocessing
- t-SNE: best for visualizing clusters
- UMAP: best for large datasets and balanced structure

11 Parameter Analysis

11.1 Perplexity in t-SNE

Perplexity controls how many neighbors each point considers.

Small perplexity values focus on local structure and create many small clusters.

Large perplexity values consider more neighbors and reveal larger patterns in the data.

Typical values range from 5 to 50.

11.2 n_neighbors in UMAP

This parameter controls the size of the neighborhood used when building the graph.
Small values focus on local structure.
Large values capture more global structure.

11.3 min_dist in UMAP

This parameter controls how tightly points are packed together.
Small values produce tight clusters.
Large values spread clusters farther apart.